

LAPORAN MINI PROJECT

MOTOMAINTENANCE: SISTEM MANAJEMEN MAINTENANCE MOTOR



Oleh:

I NYOMAN SWARDI JAYA PUTRA

F1D02310057

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MATARAM

MEI 2026

A. Deskripsi Aplikasi

MotorMaintenance adalah aplikasi manajemen perawatan sepeda motor berbasis desktop yang dirancang khusus untuk membantu pemilik kendaraan memantau kondisi teknis dan pengeluaran operasional secara *real-time*.

Aplikasi ini mengintegrasikan pencatatan aktivitas harian (seperti jarak tempuh dan konsumsi BBM) dengan sistem pemantauan kesehatan komponen (seperti oli mesin, oli gardan, ban, dan kampas rem) melalui indikator visual yang intuitif. Dengan menggunakan pendekatan berbasis data, MotorMaintenance mampu memberikan peringatan dini sebelum komponen mencapai batas usia pakainya, sehingga mencegah kerusakan fatal dan menjaga performa kendaraan tetap optimal.

B. Tujuan Project

Aplikasi ini dikembangkan dengan tujuan sebagai berikut:

1. Membantu pengguna yang belum memahami perawatan mesin (termasuk memudahkan pengguna perempuan) agar memiliki kontrol penuh atas jadwal servis motor mereka melalui indikator kesehatan komponen yang intuitif.
2. Mencegah pembengkakan biaya perbaikan besar akibat keterlambatan servis. Dengan perawatan preventif (tepat waktu), komponen motor menjadi lebih awet dan konsumsi BBM lebih terkendali.
3. Membantu pengguna memantau total pengeluaran bensin dan biaya perawatan setiap bulannya agar perencanaan keuangan pribadi tetap stabil.

C. Struktur Project dan Penerapan SoC



Berdasarkan penerapan SOC struktur project ini dibagi menjadi empat layer utama untuk memastikan kode tetap rapi, mudah dikelola, dan tidak saling tumpang tindih.

1. *UI Layer (User Interface)*

Layer ini bertugas dalam pembuatan ui atau tampilan dari sistem MotoMaintenance.

- a. Isi Folder ui/: Berisi file seperti main_window.py, panel_status.py, dialog_aktivitas.py, dan dialog_maintenance.py.
- b. Penerapan SoC: Layer ini hanya bertugas mengatur tata letak (layout), menangani input dari tombol, dan menampilkan data ke dalam widget atau tabel. UI Layer tidak boleh berisi rumus perhitungan matematika atau kueri database langsung.

2. *Logic Layer*

Layer ini bertugas dalam pembuatan logic yang akan digunakan sebagai perhitungan untuk sistem.

- a. Isi Folder logic/: Berisi file maintenance_logic.py.
- b. Penerapan SoC: Di sinilah perhitungan dilakukan, seperti membandingkan jarak tempuh (KM) saat ini dengan batas servis untuk menentukan persentase kesehatan komponen. Logic layer menerima data dari database dan mengolahnya sebelum dikirim ke UI untuk ditampilkan.

3. *Data Layer (Database)*

Layer ini bertugas atas penyimpanan dan pengambilan data dari file database.

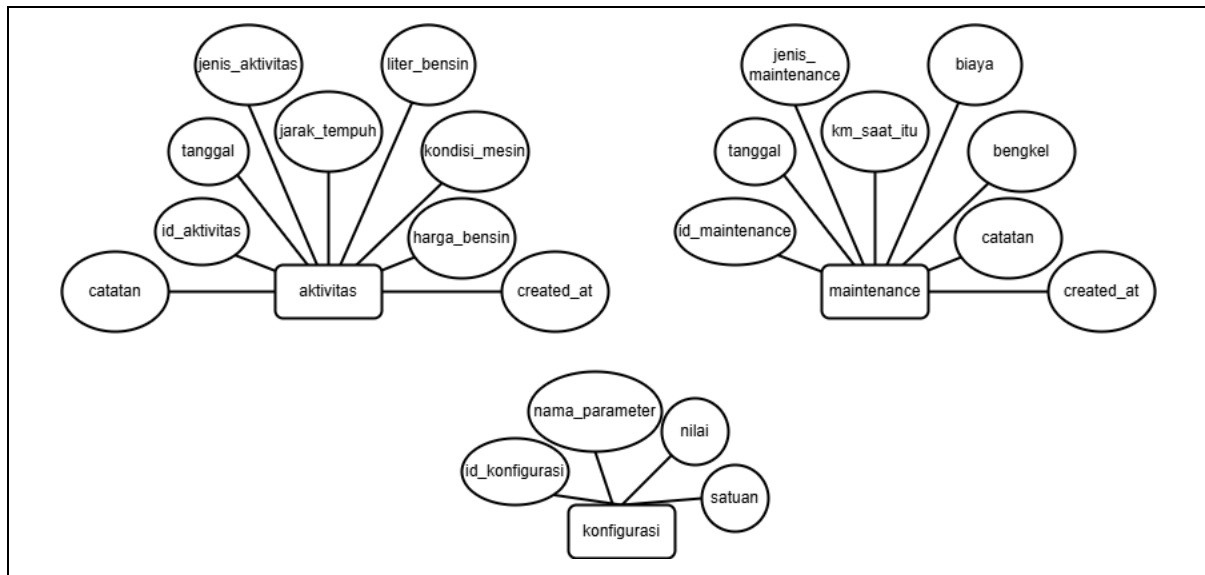
- a. Isi Folder database/: Berisi file db_manager.py dan file database motor_maintenance.db.
- b. Penerapan SoC: Layer ini berisi fungsi-fungsi CRUD (Create, Read, Update, Delete) menggunakan SQLite. Bagian UI tidak perlu tahu bagaimana cara database menyimpan data; UI cukup memanggil fungsi dari Data Layer untuk mendapatkan hasil yang dibutuhkan.

4. *Style Layer*

Layer ini bertugas dalam pembuatan style ui sehingga tampilan yang diberikan kepada pengguna terlihat lebih menarik dan rapi

- a. Isi Folder style/: Berisi file app.qss.
- b. Penerapan SoC: Dengan memisahkan gaya ke dalam file QSS (CSS untuk Qt), kita bisa mengubah seluruh tema warna aplikasi (misalnya dari tema ungu ke tema classic merah-biru) tanpa harus mengubah logika program di file .py.

D. Desain Database



E. Fitur Utama dan Cara Kerjanya

1. **Aktivitas Harian (CRUD):** Menambahkan aktivitas harian yang digunakan sebagai tolak ukur dalam memantau penggunaan motor secara *real-time*. Fitur ini mencatat jarak tempuh (KM), konsumsi bahan bakar (Liter), dan biaya bensin.

Cara Kerja: Setiap kali pengguna selesai berkendara sehari-hari, pengguna akan memasukkan data aktivitas tersebut ke sistem. Sistem kemudian secara otomatis mengakumulasi total jarak tempuh yang akan menjadi dasar perhitungan masa pakai komponen mesin tanpa pengguna harus menghitungnya secara manual.

2. **Status Kesehatan Komponen (Monitoring):** Menampilkan kondisi kesehatan komponen vital motor seperti oli mesin, oli gardan ban, dan rem melalui indikator visual.

Cara Kerja: Sistem membandingkan data kilometer terakhir saat servis dengan total kilometer saat ini. Hasilnya disajikan dalam bentuk *progress bar* berwarna (Hijau/Kuning/Merah). Ini memudahkan pengguna awam untuk mengetahui kapan harus ke bengkel hanya dengan melihat warna, tanpa perlu mengecek fisik mesin secara langsung.

3. **Riwayat Maintenance (History):** Menyimpan catatan lengkap setiap kali motor masuk ke bengkel, termasuk detail biaya dan jenis perbaikan yang dilakukan.

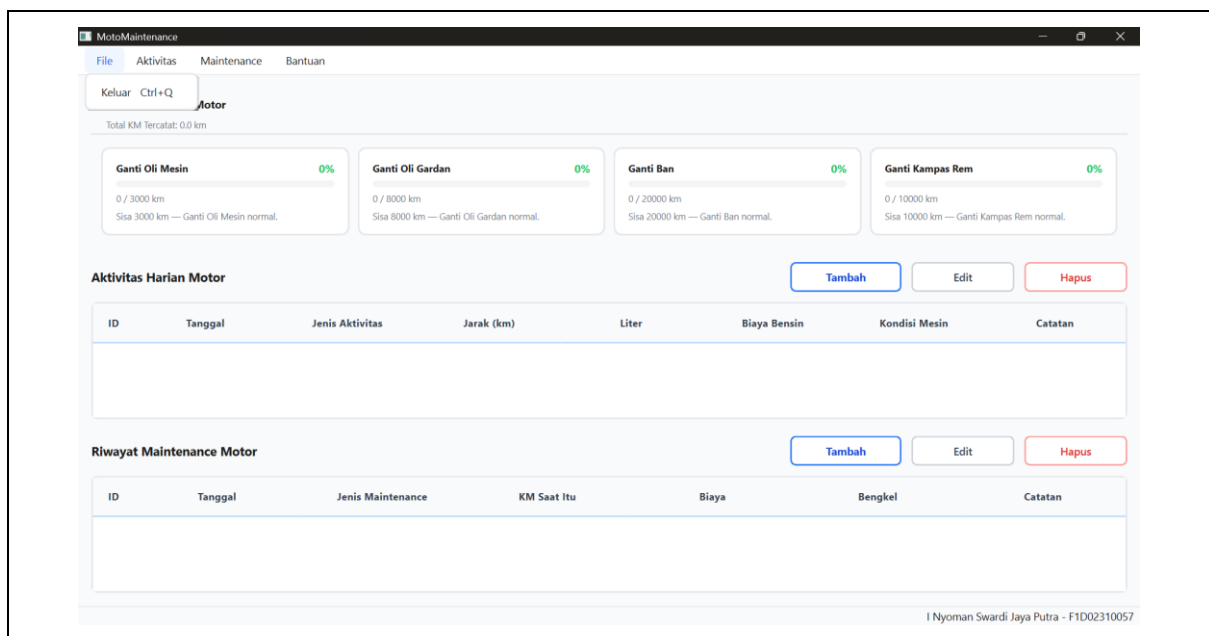
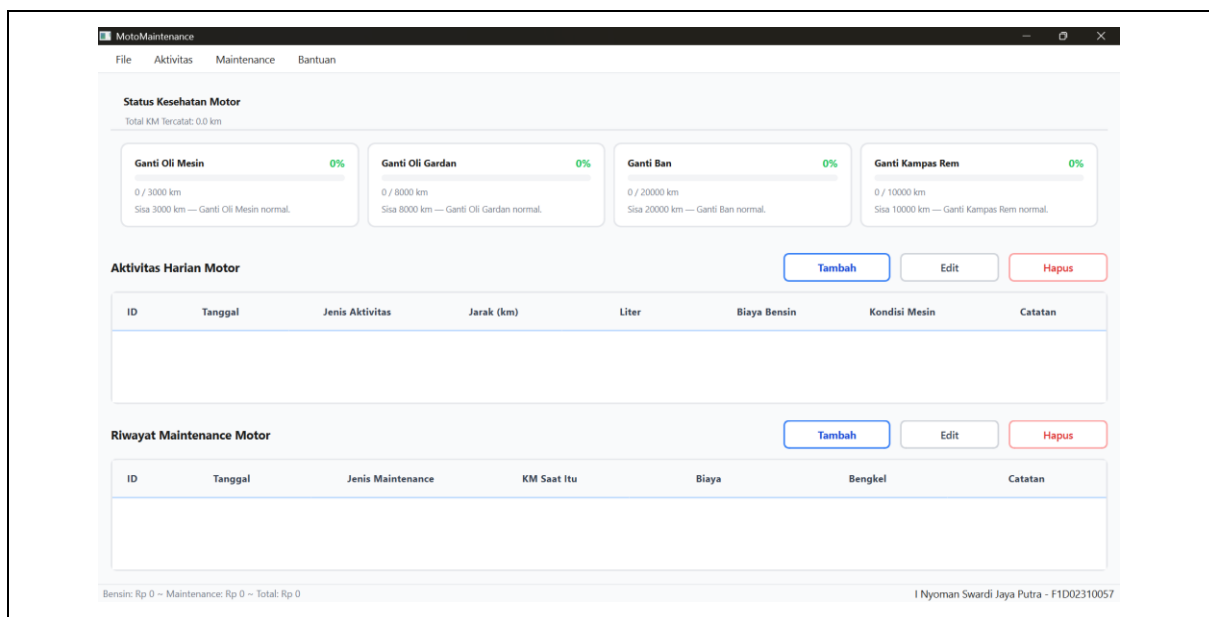
Cara Kerja: Pengguna akan memasukkan riwayat maintenance ke sistem setiap kali melakukan servis motor, di mana data servis akan disimpan secara permanen untuk memberikan transparansi biaya. Pengguna dapat melihat pengeluaran bulanan dan memastikan bahwa komponen diganti pada interval yang benar, sehingga mencegah

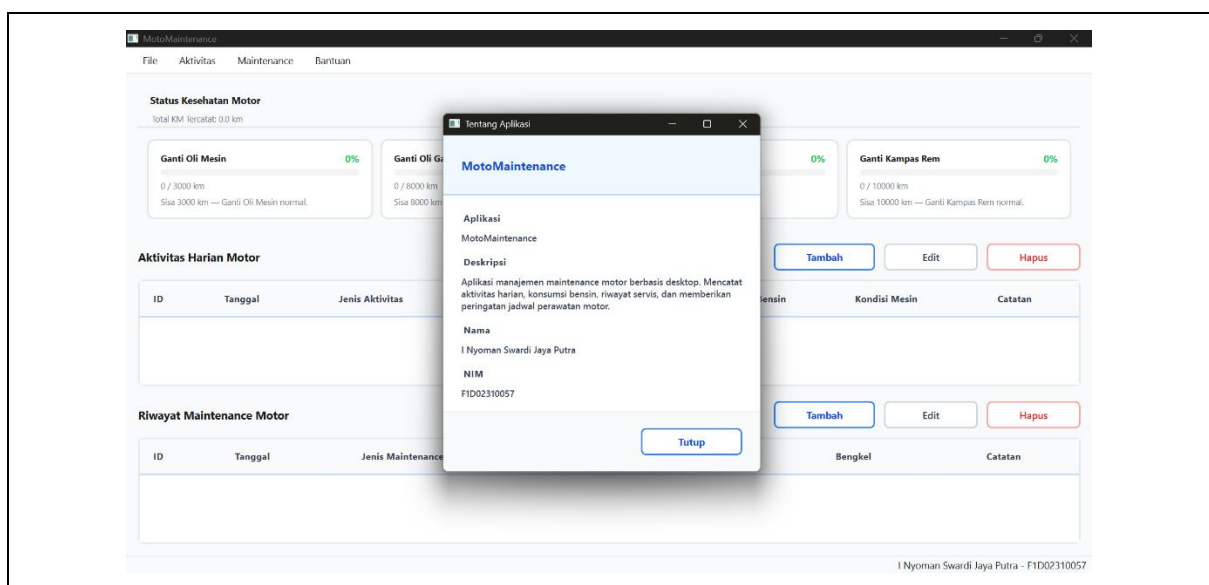
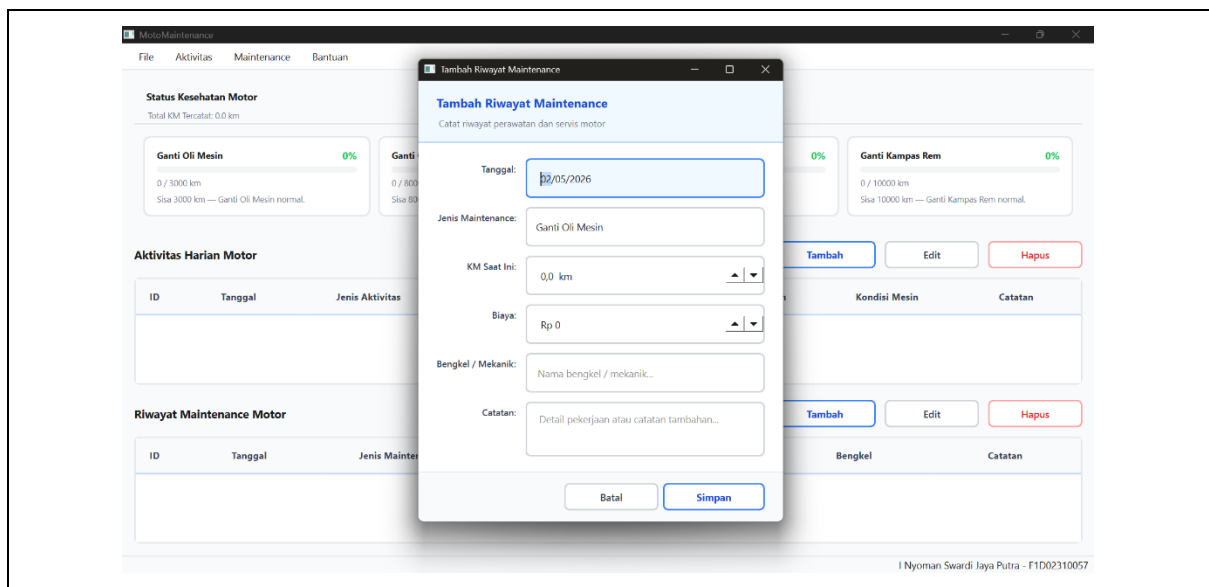
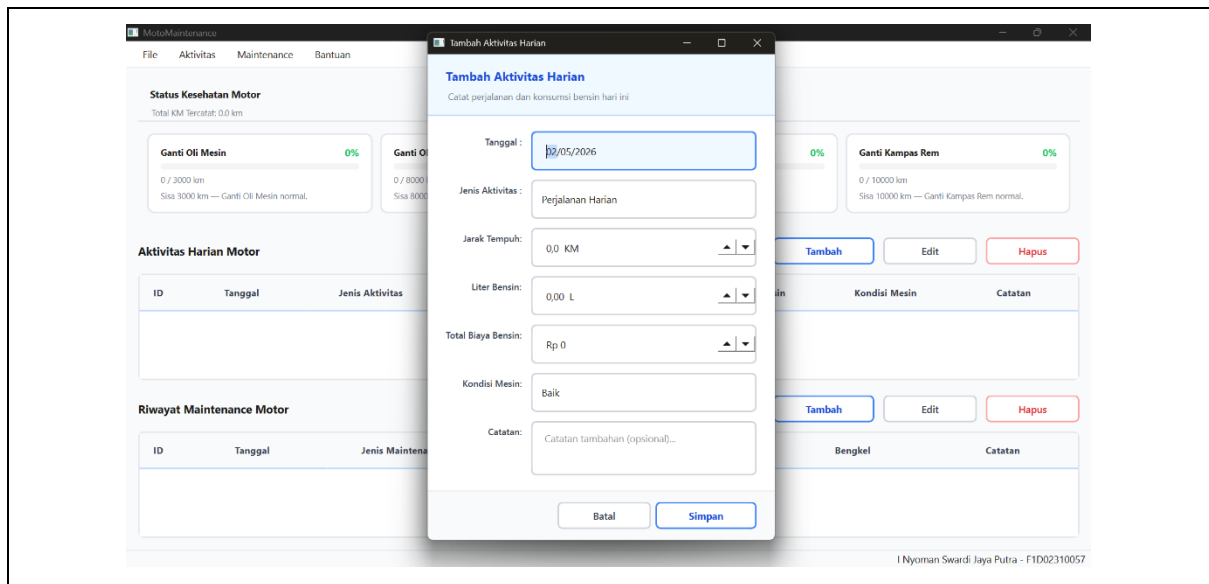
praktik "curang" atau penggantian komponen yang sebenarnya masih layak pakai di bengkel.

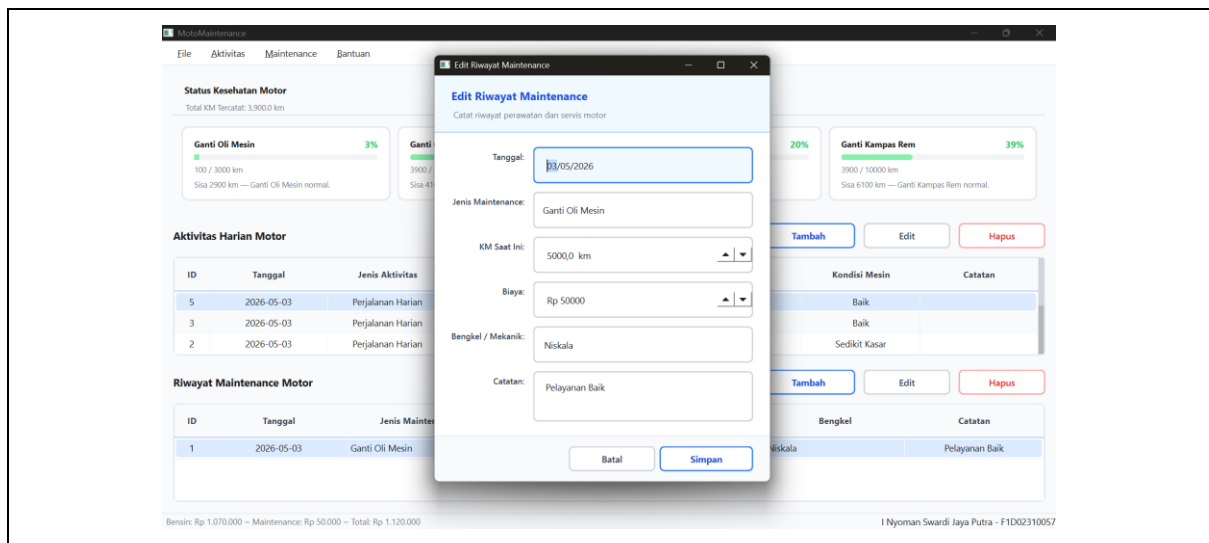
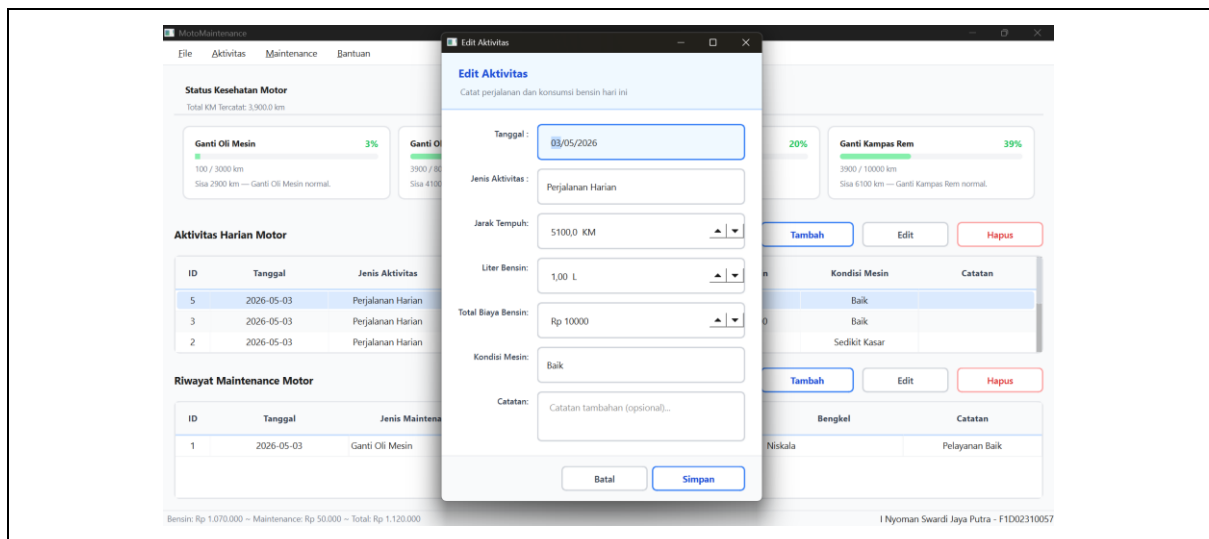
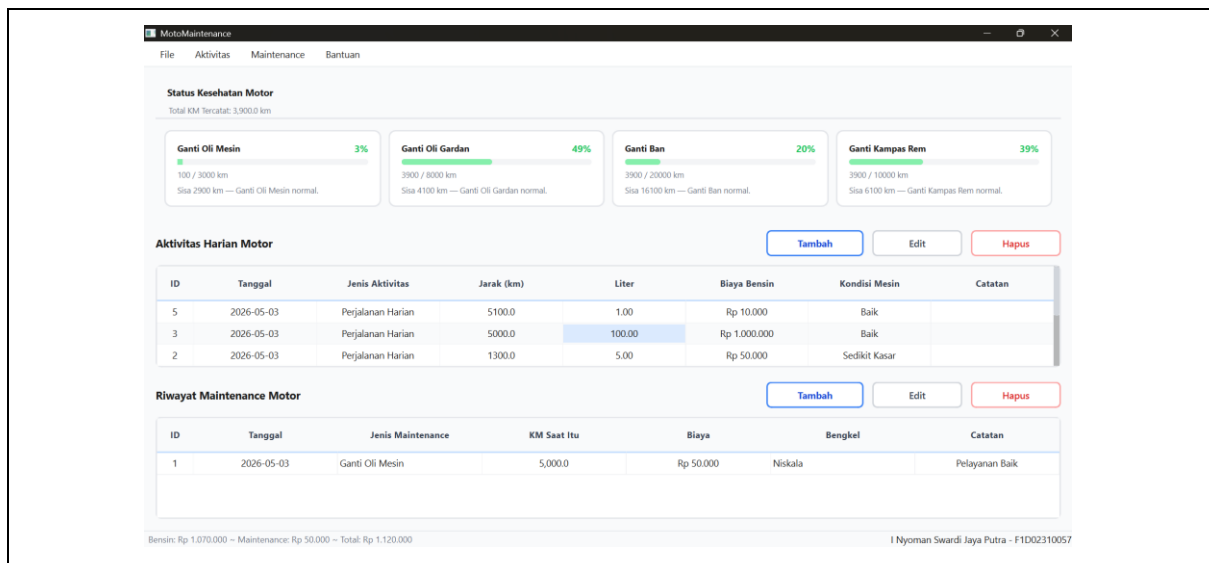
4. Ringkasan Keuangan (Financial Tracking): Menghitung total pengeluaran operasional kendaraan (Bensin + Servis) dalam periode satu bulan.

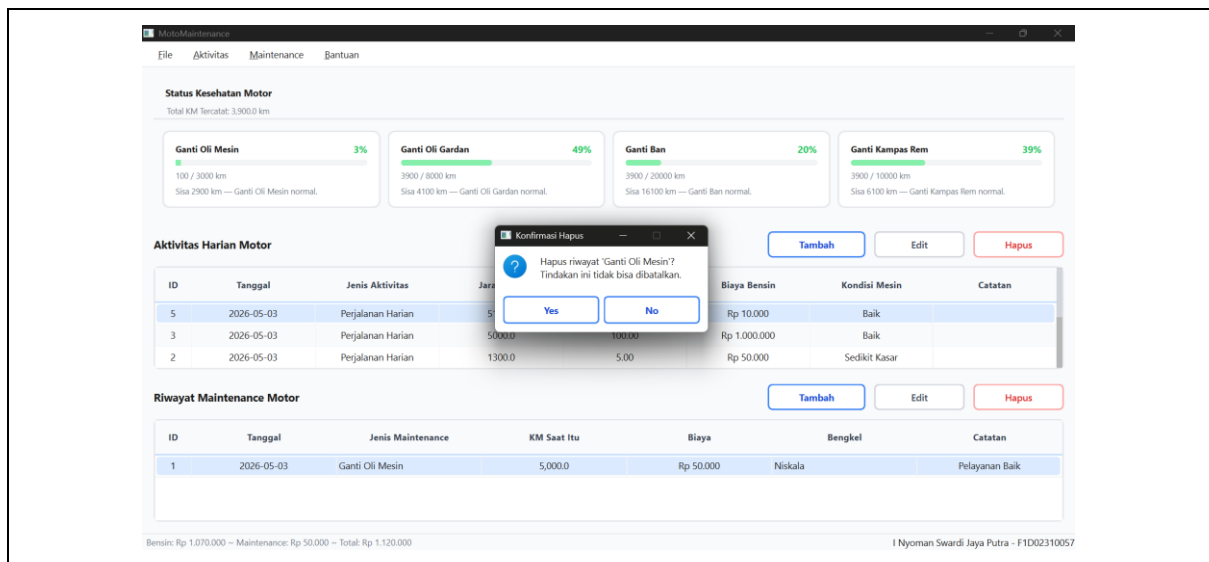
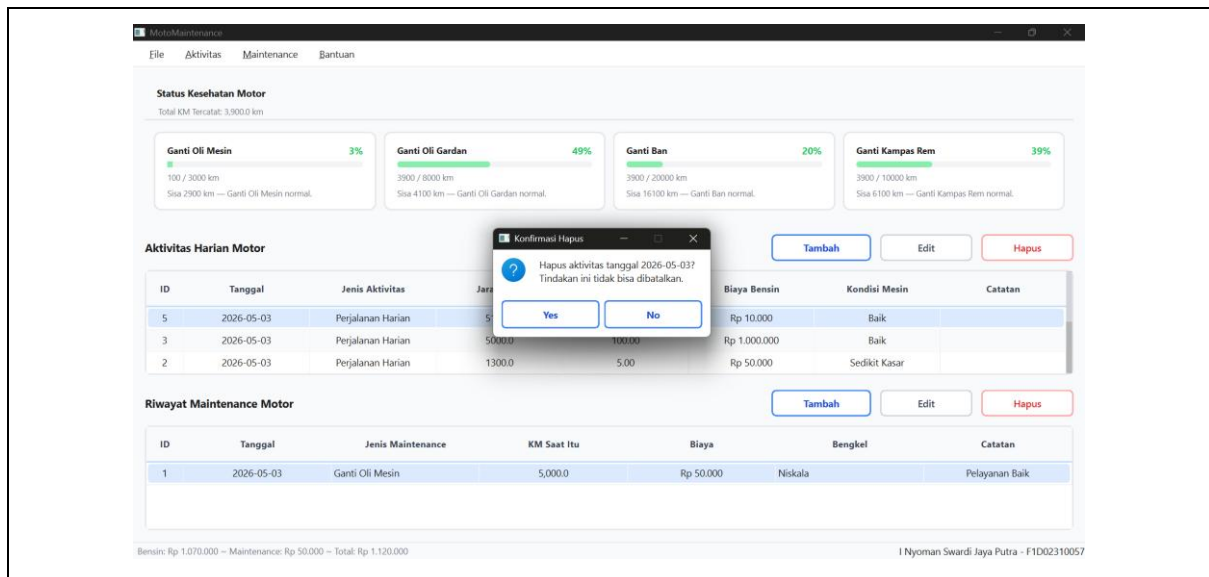
Cara Kerja: Fitur ini mengagregasi seluruh biaya yang diinput dalam fitur Aktivitas dan Maintenance, lalu menampilkannya di bagian bawah panel utama. Hal ini bertujuan agar pengguna (terutama yang sedang mengelola anggaran ketat) dapat melihat efisiensi biaya perawatan motor mereka dengan jelas.

F. Screenshot Antarmuka









G. Link YouTube

<https://youtu.be/bw89kdNh-E8>

H. Link Github

<https://github.com/inyomanswardijayaputra/pv26-miniproject-motomaintenance-f1d02310057>